计算机组成与结构
—存储系统3

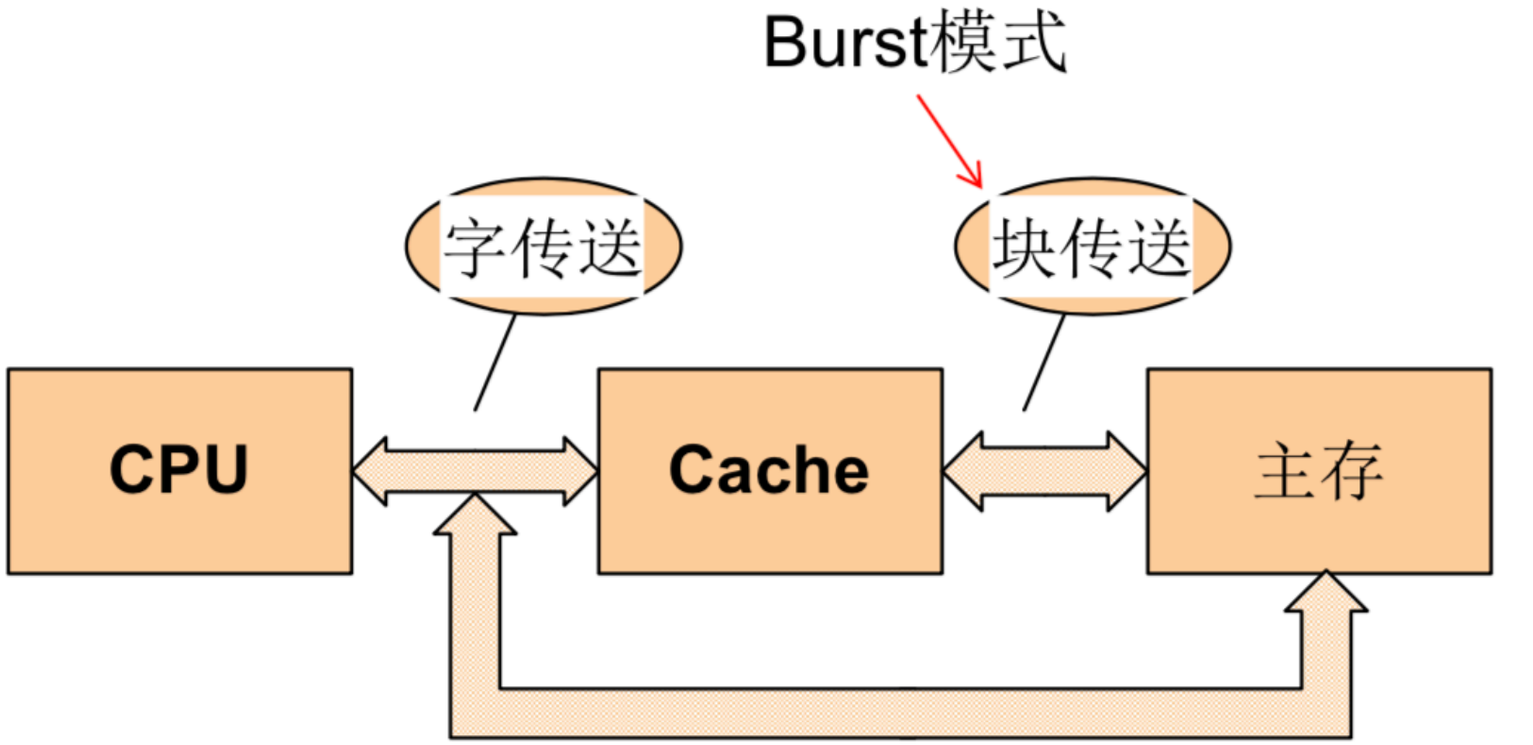
李瑞　　教授
蒋志平　讲师
计算机科学与技术学院

温故

- 概述6264芯片特点
- 简述8086系统内存拼接的特殊设计
- 简述2164芯片特点及其地址输入方法
- 简述多端口存储器？大概原理?
- 简述多体交叉存储，其原理?
- 简述相联存储，其原理?
- 简述Cache的工作原理？基于什么原理?
- 简述存储系统？有哪两类？目的是什么?
- 简述全相联映射？ 优缺点?
- 简述直接映射？ 优缺点?

How Cache works?

How it works?



1. Cache的工作过程

- 整块缓存、单字输出
- **Cache-**处理机通路

  主存-处理机通路
- 流水处理：地址变换、访问**Cache**

主存地址（来自CPU）

块号B | 块内地址W

主存地址

相联存储表

主存→Cache地址变换

未命中

命中

Cache地址

已满

未满

块号b | 块内地址w

Cache替换策略

高速存储器

替换块

装入块

Cache

主存储器

数据送CPU

主存与Cache的地址映射

- 第一步：地址映象：

  把主存中的数据按照某种规则装入到**Cache**中，并建立主存地址与**Cache**地址之间的对应关系。

- 第二步：地址变换：

  当主存数据已经装入到**Cache**之后，在程序运行过程中，把主存地址变换成**Cache**地址。

- 在选取地址映象方法要考虑的主要因素：
  - 地址变换的硬件高速、低价、易于实现；
  - **Cache**空间利用率要高；
  - 发生块冲突的概率要小。
- 以“块”为单位进行调度。

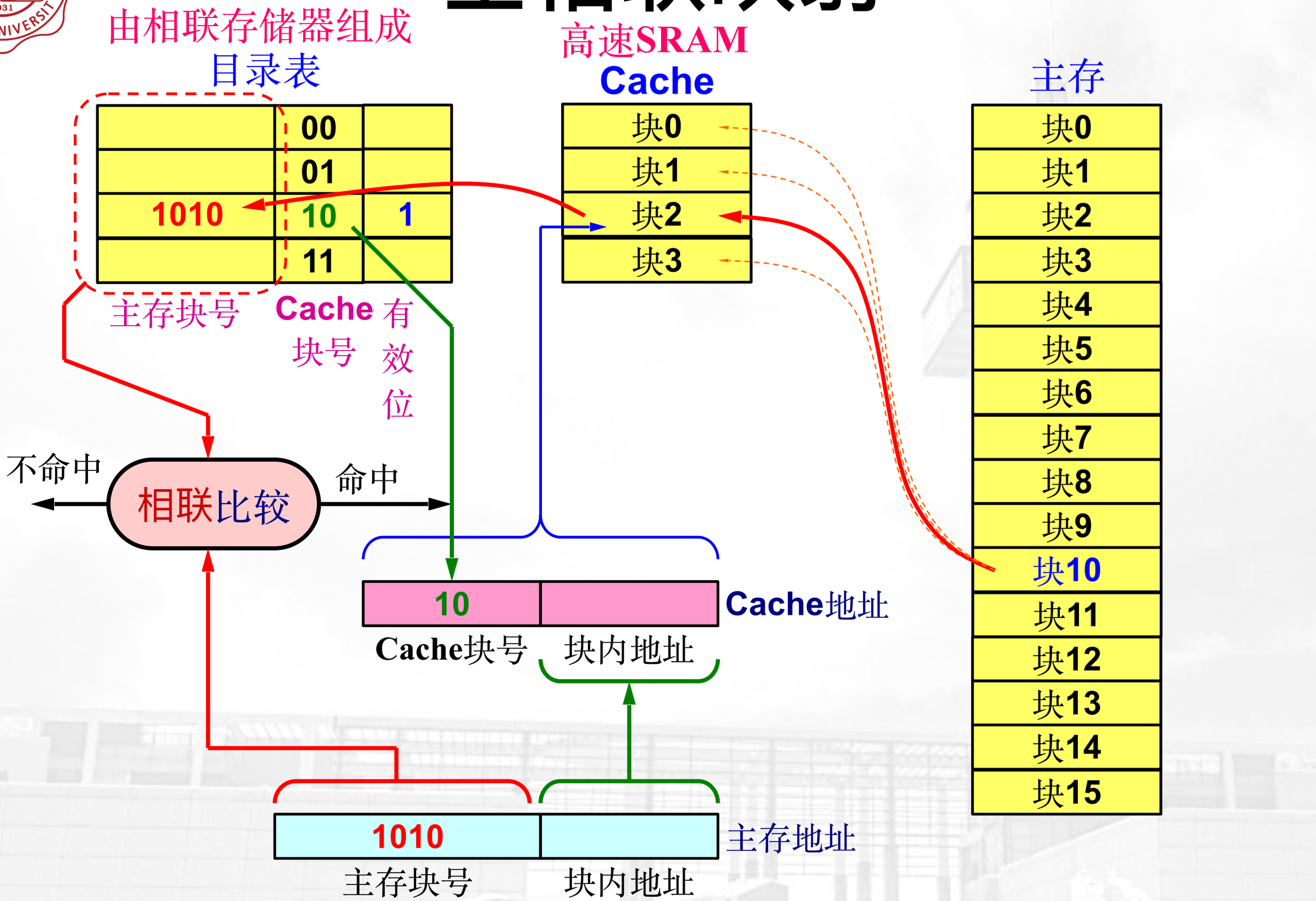
- 全相联
- 直接映射
- 组相联

全相联映射

- **映射规则：**

  主存的任意一块可以映象到Cache中的任意一块。

全相联映射


由相联存储器组成
目录表
高速SRAM
Cache
主存
00
01
1010
10
1
11
块0
块1
块2
块3
块0
块1
块2
块3
块4
块5
块6
块7
块8
块9
块10
块11
块12
块13
块14
块15
主存块号
Cache
块号
有
效
位
不命中
相联比较
命中
10
Cache地址
Cache块号
块内地址
1010
主存地址
主存块号
块内地址

全相联映射

- **映射规则：**

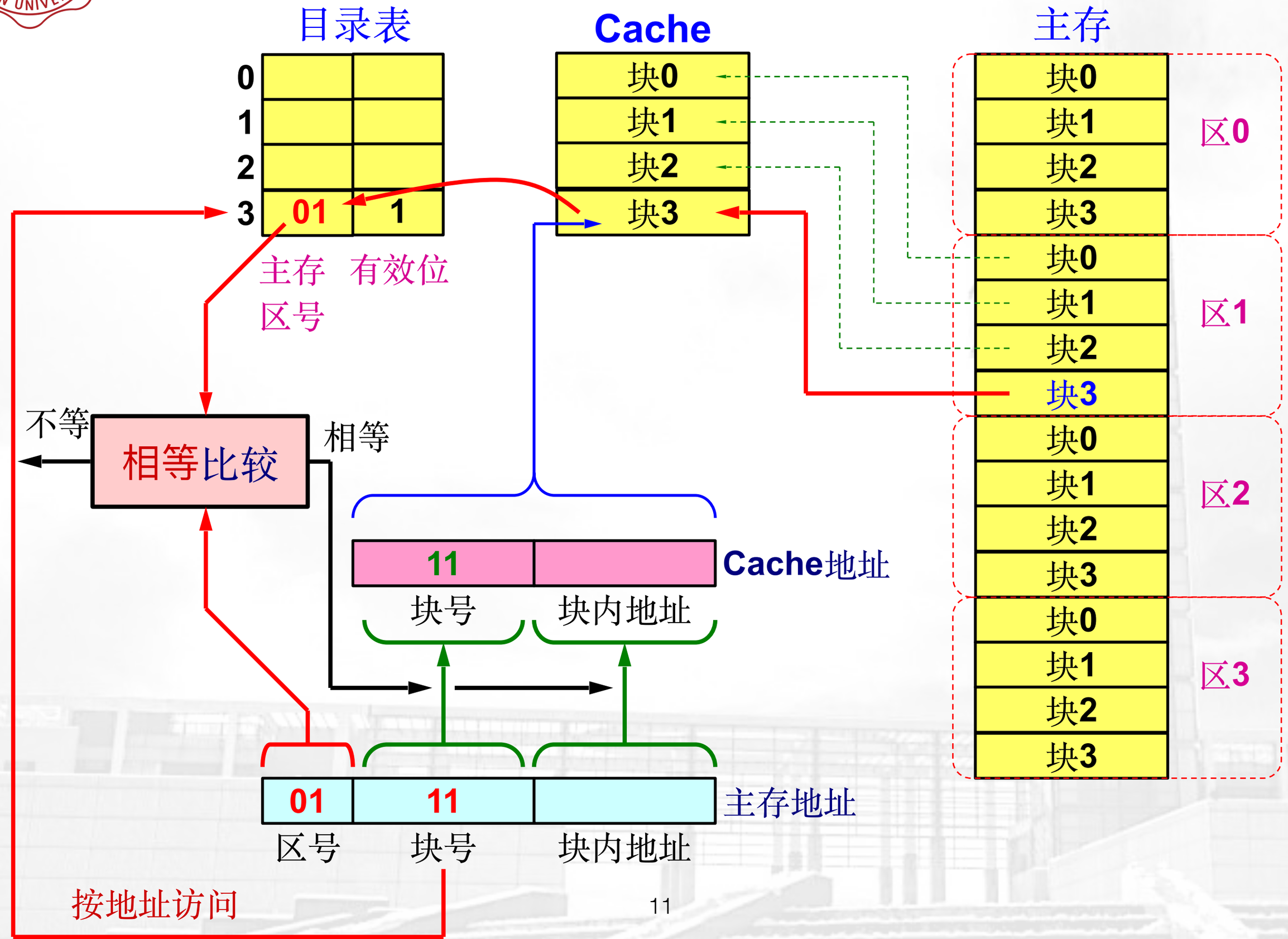
  主存的任意一块可以映象到Cache中的任意一块。

- **特点：**
  - **块冲突概率低，Cache空间利用率高。**
  - **相联目录表容量大→成本高、查表速度慢。**
    - **why??**

直接映射

- 映射规则：主存的每一块只能映象到Cache的一个特定的块中。
- 整个Cache地址与主存地址的低位部分完全相同

直接映射


目录表
Cache
主存
0
1
2
3
01
1
主存 有效位
区号
块0
块1
块2
块3
区0
区1
区2
区3
不等
相等比较
相等
11
Cache地址
块号
块内地址
01
11
主存地址
区号
块号
块内地址
按地址访问

直接映射

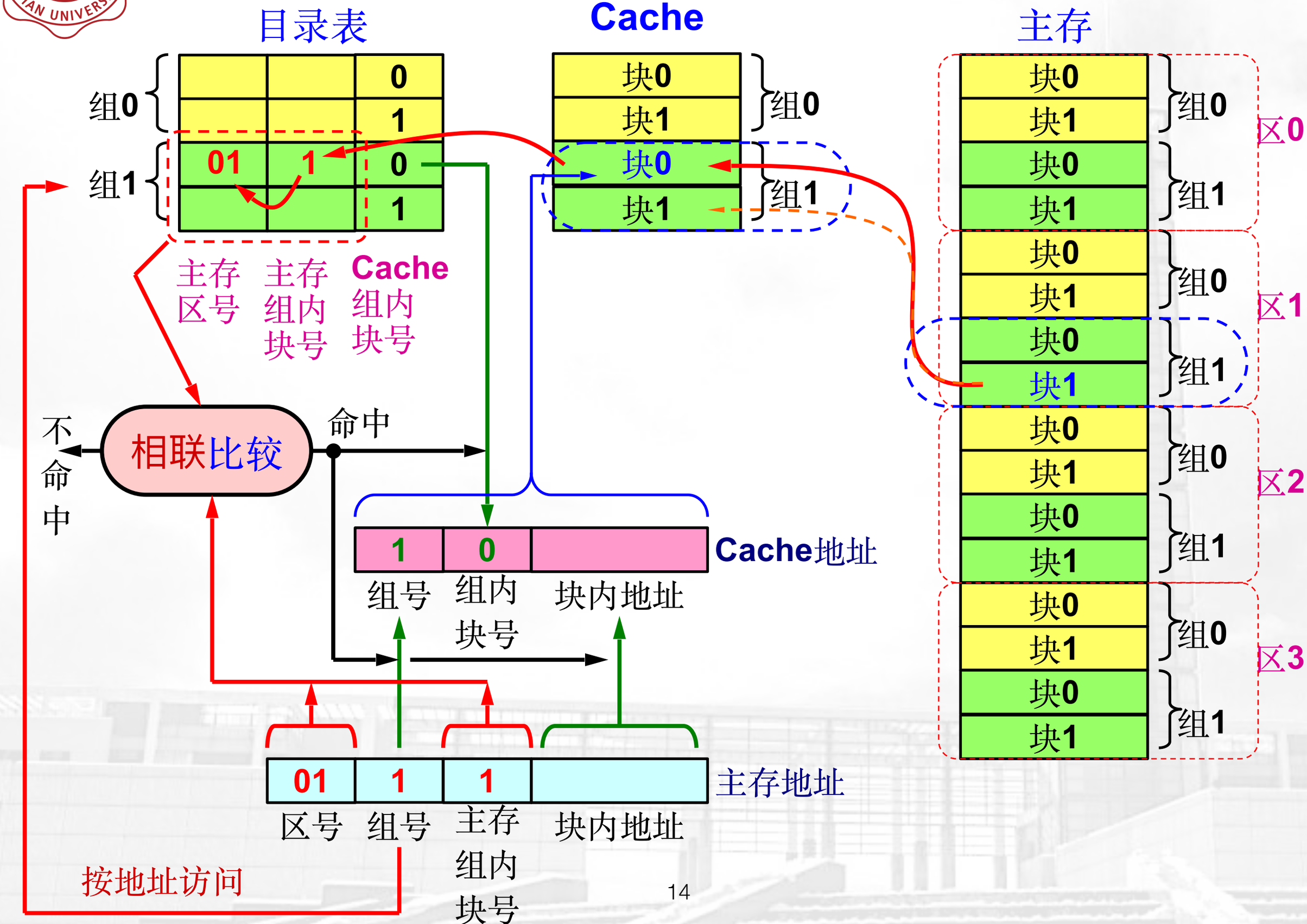
- 映射规则：主存的每一块只能映象到Cache的一个特定的块中。
- 整个Cache地址与主存地址的低位部分完全相同。
- 优点：
  - 硬件简单，不需要相联存储器，只要
    - 容量较小的按地址访问的区号标志表存储器；
    - 少量外比较电路。
  - 访问速度快（无需地址变换）。
- 缺点：
  - Cache块冲突概率高；
  - Cache空间利用率很低。

组相联映射

- 目的：减小相联目录表的容量，降低成本，提高地址变换速度。
- 映射规则：将主存和Cache都机械等分成相同大小的块，并将主存空间按照Cache大小等分成区，再将Cache和主存中的每一区都等分成大小相同的组，让主存各区中的某组中的任何一块均可直接映象装入到Cache中对应组的任何一块上。

从主存的组到Cache的组之间采用直接映象方式，在两个对应的组内部采用全相联映象方式 ——组间直接，组内全相联。

组相联映射


目录表
Cache
主存
组0
组1
0
1
01
1
0
1
块0
块1
块0
块1
组0
组1
区0
区1
区2
区3
主存区号
主存组内块号
Cache组内块号
相联比较
不命中
命中
1
0
Cache地址
组号
组内块号
块内地址
01
1
1
主存地址
区号
组号
主存组内块号
块内地址
按地址访问

组相联映射

- 优点：
  - 块的冲突概率比较低；
  - 块的利用率大幅度提高；
  - 块的失效率明显降低。
- 缺点：

  实现难度和造价比直接映射方式高。

[例1] 直接映射方式的Cache，初始内容为空。

Cache容量为2KB，以128B分块。主存容量为1MB，求：

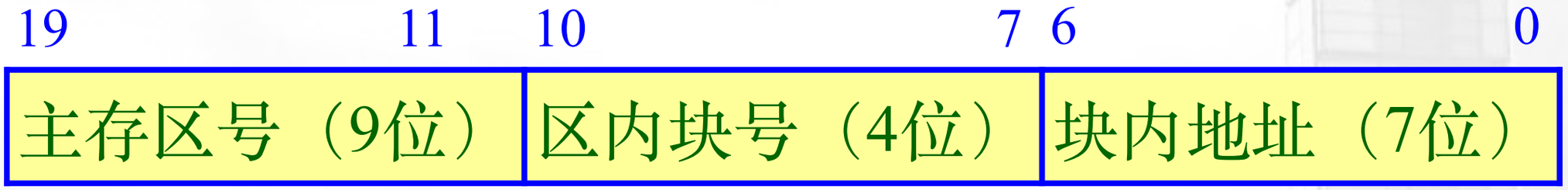
(1) Cache分为多少块？Cache块号及块内地址各为多少位？为实现地址映像，主存地址应如何划分？

(2) 主存中顺序排列的第129块（即若主存不分区，以第0块开始顺序排列时）内容调入到Cache的哪一块中？

(3) 在上一步的基础上，读写主存地址328CBH时是否命中？

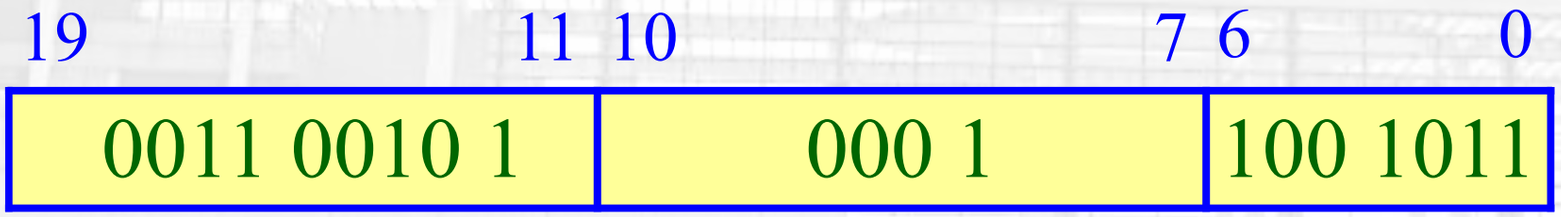
【解】

(1) Cache：2KB/128B = 16块，主存8K块，共512个区。



(2) 129÷16 = 8余1，即主存第8区的第1块，因此应装入Cache的第1块。

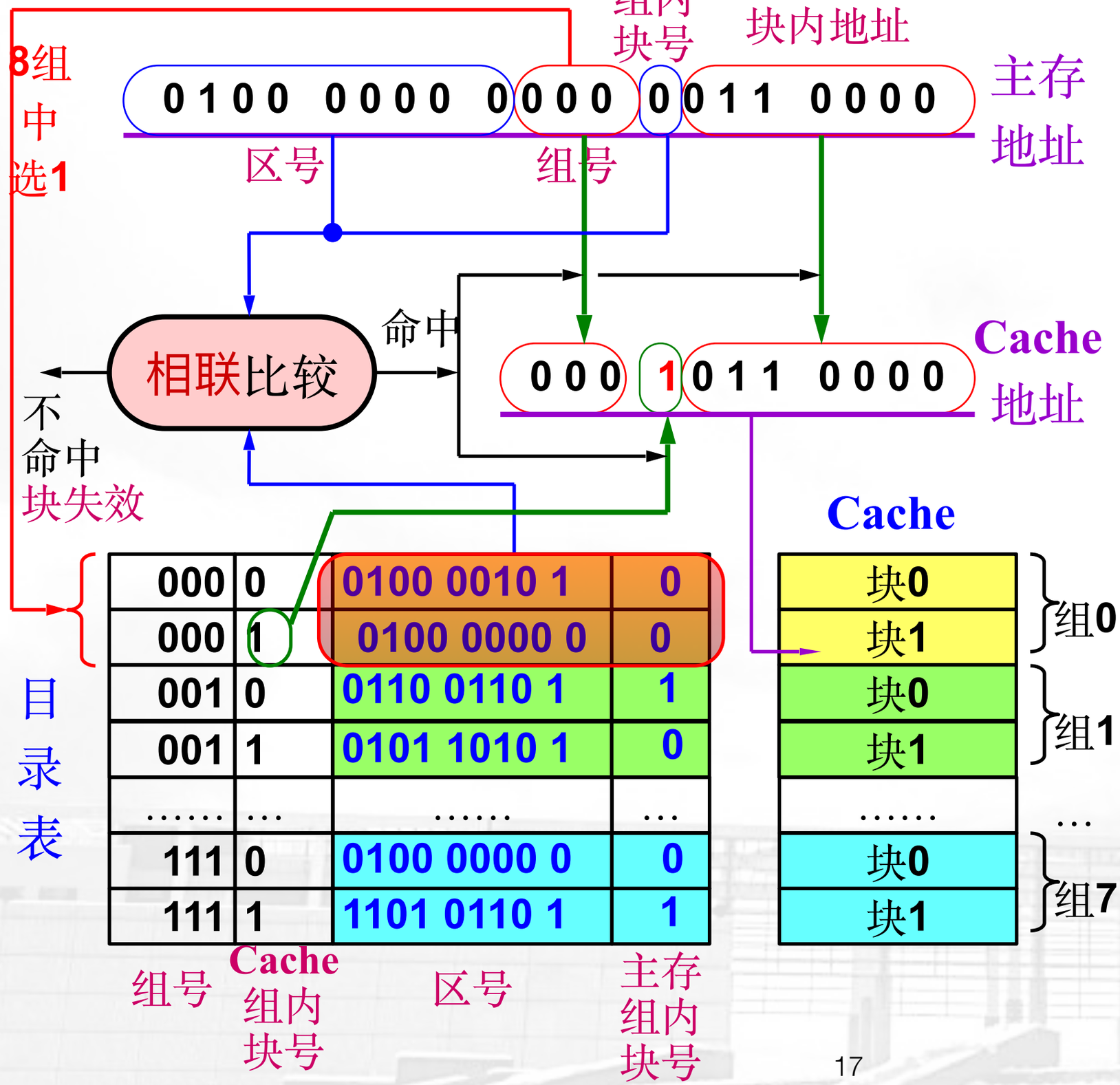
(3) 虽然也对应Cache的第1块，但主存区号不同，不命中。



[例2] 组相连：主存1MB；Cache 2KB，128B为一块，一组2块。

→ Cache 16块，共8组；主存8K块，共512个区(每区8组)。

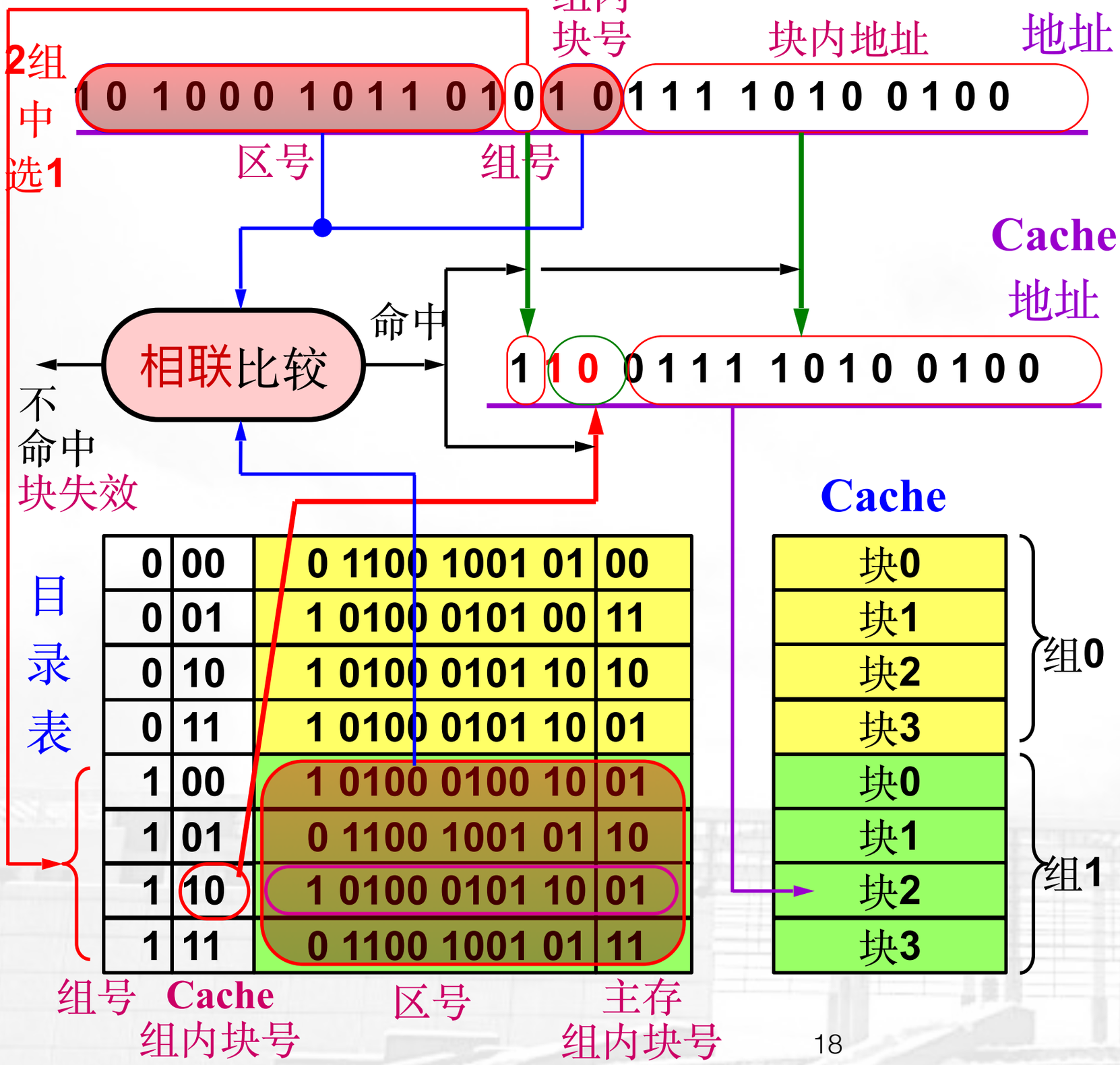
主存地址：40030H

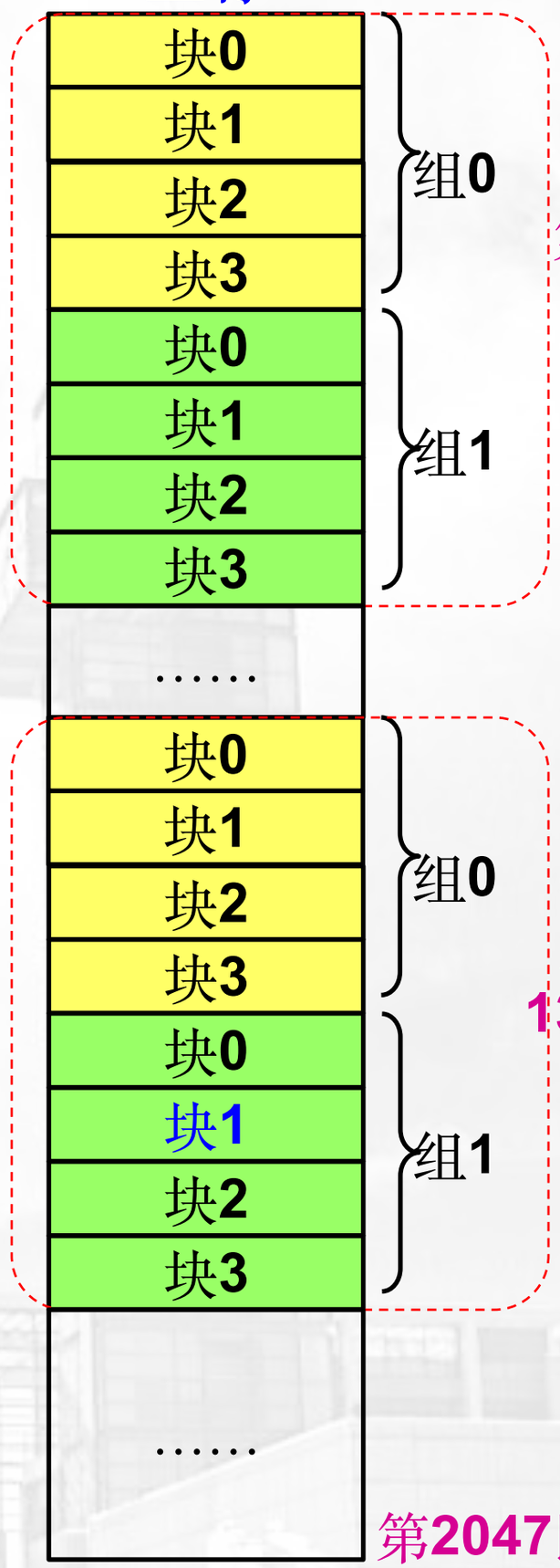


[例3] 组相连：主存64MB；Cache 32KB，4KB为一块，一组4块。

→ Cache 8块，共2组；主存16K块，共2K个区(每区2组)。

主存地址：28B57A4H





往年考题

四.（本题 15 分）某计算机系统的内存容量为 4GB，Cache 容量为 256KB，均采用字节编址。Cache 与内存采用 8 路组相联映射，Cache 每块为 4KB。

请问：

1. 内存和 Cache 地址的各字段分别有多少位？
2. 每次地址变换时，参与相联比较的位数是多少？
3. 若当前地址变换表的部分有效内容如下（有效位为 1，表示 Cache 块有效）。当 CPU 访问内存地址分别为 01234567H 和 FEDCBA98H 时，请问是否能够命中，若命中则给出相应的 Cache 地址。

地址	内存区号	组内块号	有效位
000 000	0009 H	000B	1
001 011	3FD7 H	010B	1
001 110	2440 H	101B	1
011 001	3FD7 H	011B	1
011 110	076E H	111B	1
110 000	0048 H	100B	1
110 100	0009 H	000B	1
111 111	0048 H	100B	1

替换策略

主存地址（来自CPU）

块号B | 块内地址W

主存地址

相联存储表

主存→Cache地址变换

未命中

命中

Cache地址

已满

未满

块号b | 块内地址w

Cache替换策略

高速存储器

替换块

装入块

Cache

主存储器

数据送CPU

替换算法

直接映射的**cache**中，由于某个主存块只与一个**cache**字块有映射关系，因此替换策略很简单。而在全相连和组相连中，主存块可以写入**cache**中若干位置，这就有一个选择替换掉哪一个**cache**字块的问题，即所谓替换算法问题。

全相联、组相联：

■随机算法（**RAND, Random Algorithm**）

■先进先出算法 (**FIFO, First-in First-out Algorithm**)

- 利用了历史信息
- 没有反映局部性——最先调入的，可能也是要使用的。

■最不经常使用算法(**LFU, Least Frequently Used Algorithm**)：计数器位数多，实现困难。

■近期最少使用算法(**LRU, Least Recently Used Algorithm**)：多/少⇨有/无。随机期法、定期法。

■最佳替换算法(**OPT, Optimal replacement algorithm,** 作弊~~)

替换算法

FIFO策略

访问顺序	1	2	3	4	5	6	7	8
地址块号	2	11	2	9	7	6	4	3
块分配情况	2	2	2	2	2	6	6	6
	–	11	11	11	11	11	4	4
	–	–	–	9	9	9	9	3
	–	–	–	–	7	7	7	7
操作状态	调进	调进	命中	调进	调进	替换	替换	替换

替换算法

FIFO策略导致的“颠簸”

访问顺序	1	2	3	4	5	6	7	8
地址块号	2	11	9	7	6	2	11	9
块分配情况	2	2	2	2	6	6	6	6
	–	11	11	11	11	2	2	2
	–	–	9	9	9	9	11	11
	–	–	–	7	7	7	7	9
操作状态	调进	调进	调进	调进	替换	替换	替换	替换

替换算法

LRU策略，将最近4次最少使用的块替换

访问顺序	1	2	3	4	5	6	7	8
地址块号	2	11	2	9	7	6	4	3
块分配情况	2	2	2	2	2	2*	4	4
	–	11	11	11	11*	6	6	6
	–	–	–	9	9	9	9*	3
	–	–	–	–	7	7	7	7*
操作状态	调进	调进	命中	调进	调进	替换	替换	替换

替换算法

比较FIFO和LRU：Cache 3块，程序在内存中占5块，第一次执行。

时间	1	2	3	4	5	6	7	8	9	10	11	12
所执行程序块号	P2	P3	P2	P1	P5	P2	P4	P5	P3	P2	P5	P2
FIFO： 命中率： 3/12＝25％	P2	P2	P2	P2	P5	P5	P5	P5	P3	P3	P3	P3
		P3	P3	P3	P3	P2	P2	P2	P2	P2	P5	P5
				P1	P1	P1	P4	P4	P4	P4	P4	P2
LRU： 命中率： 5/12＝42％	P2	P2	P2	P2	P2	P2	P2	P2	P3	P3	P3	P3
		P3	P3	P3	P5	P5	P5	P5	P5	P5	P5	P5
				P1	P1	P1	P4	P4	P4	P2	P2	P2

替换算法

- 例：

假定程序在主存为5块，Cache为3块。CPU执行程序的顺序为：P2、P3、P2、P1、P5、P2、P4、P5、P3、P2、P5、P2。画出FIFO算法和LRU算法的命中情况如图所示。

P2	P3	P2	P1	P5	P2	P4	P5	P3	P2	P5	P2
2	2	2	2	5	5	5	5	3	3	3	3
	3	3	3	3	2	2	2	2	2	5	5
			1	1	1	4	4	4	4	4	2
		H					H		H		

P2	P3	P2	P1	P5	P2	P4	P5	P3	P2	P5	P2
2	2	2	2	2	2	2	2	3	3	3	3
	3	3	3	5	5	5	5	5	5	5	5
			1	1	1	4	4	4	2	2	2
		H			H		H			H	H

替换算法

三种**Cache**替换算法的比较：

	Associativity								
	Two-way			Four-way			Eight-way		
Size	LRU	Random	FIFO	LRU	Random	FIFO	LRU	Random	FIFO
16 KB	114.1	117.3	115.5	111.7	115.1	113.3	109.0	111.8	110.4
64 KB	103.4	104.3	103.9	102.4	102.3	103.1	99.7	100.5	100.3
256 KB	92.2	92.1	92.5	92.1	92.1	92.5	92.1	92.1	92.5

Data cache **misses per 1000 instructions** comparing least-recently used, random, and first in, first out replacement for several sizes and associativities. There is little difference between LRU and random for the largest size cache, with LRU outperforming the others for smaller caches. FIFO generally outperforms random in the smaller cache sizes. These data were collected for a block size of 64 bytes for the Alpha architecture using 10 SPEC2000 benchmarks. Five are from SPECint2000 (gap, gcc, gzip, mcf, and perl) and five are from SPECfp2000 (applu, art, equake, lucas, and swim).

数据来源于

《Computer Architecture: A Quantitative Approach》 Fourth Edition

John L. Hennessy， David A. Patterson

Cache/内存一致性问题

Cache/内存一致性问题

- 读数据是否存在一致性问题?
- 写数据如何保持Cache/Memory一致?
- 两种策略:
  - 写回法(Write Back)
  - 全写法(Write Through)

Cache/内存一致性问题

- 写回法（Write Back）
  - 当CPU写Cache命中时，只将数据写入Cache，不立即写入主存。只有当被修改的Cache块被替换时才写回主存中。
  - 当CPU写Cache未命中时，则写修改是将相应主存块调入Cache之后，在Cache中进行。对主存的修改仍留待该块替换出去时进行。
  - 用修改位(Dirty Bit)，追踪是否被CPU修改过。

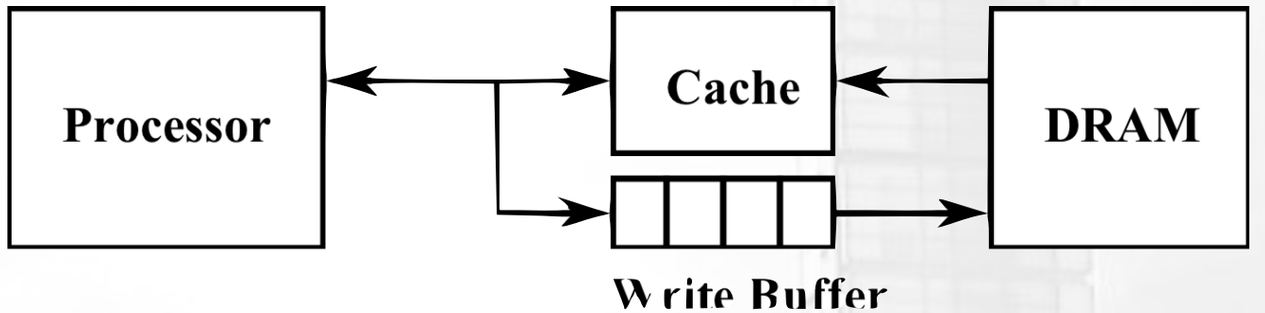
Cache/内存一致性问题

- 全写法（Write Through）
  - 当Cache命中时，在将数据写入修改Cache的同时写入修改主存，较好地保证了主存与Cache内容的一致性。
  - 当Cache未命中，直接向主存进行写入。这时可以采用两种方式进行处理：

- What a simple solution ?!

Cache/内存一致性问题

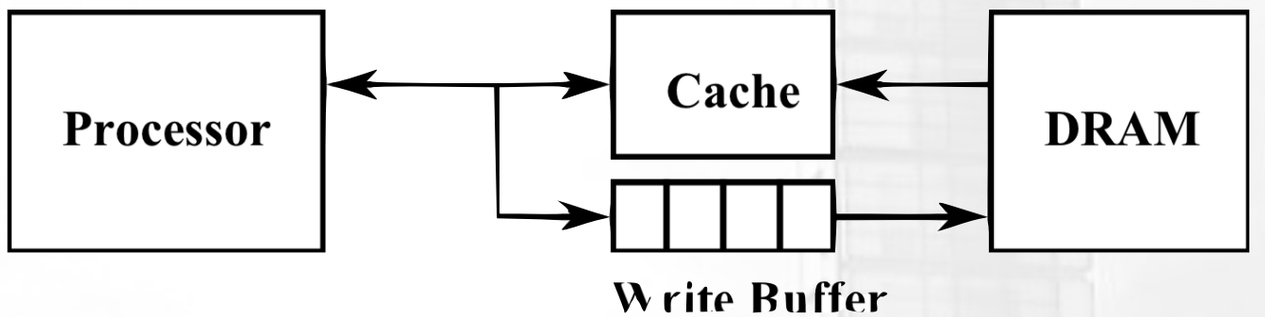
- 全写法（Write Through)
  - Cache 和内存速度不一致如何解决?
    - 速度不够Buffer凑！



- 但貌似还有问题....
  - 谁把Buffer数据搬到内存?
    - 加个控制器让内存主动取回来...

Cache/内存一致性问题

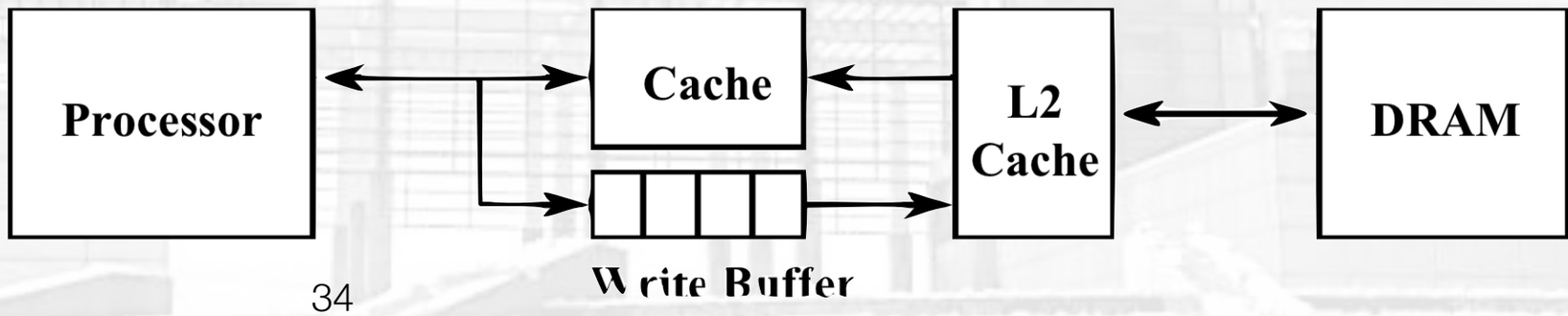
- 但貌似还有问题....
  - 谁把Buffer数据搬到内存?
    - 加个控制器让内存主动取回来...



- 但貌似还有！问题....
  - Buffer会不会爆掉?
    - 加2级Cache!

竞猜：L2 Cache的写策略是?

回写法(Write Back)!



7.3.3 主存与Cache内容的一致性问题

采用Cache的访存操作

Cache类型	操作	访存操作	访存时间
写直达Cache	读命中	只读Cache	T_C
	写命中	写Cache，同时写内存	T_C(隐藏T_M)
	读不命中	调入Cache块，再读Cache	$T_B + T_C$
	写不命中	只写内存	T_M
写回Cache	读命中	只读Cache	T_C
	写命中	只写Cache	T_C
	读不命中	调入Cache块，再读Cache	$T_B + T_C$
	写不命中	调入Cache块，再写Cache	$T_B + T_C$

Cache性能分析与优化

Cache性能分析与优化

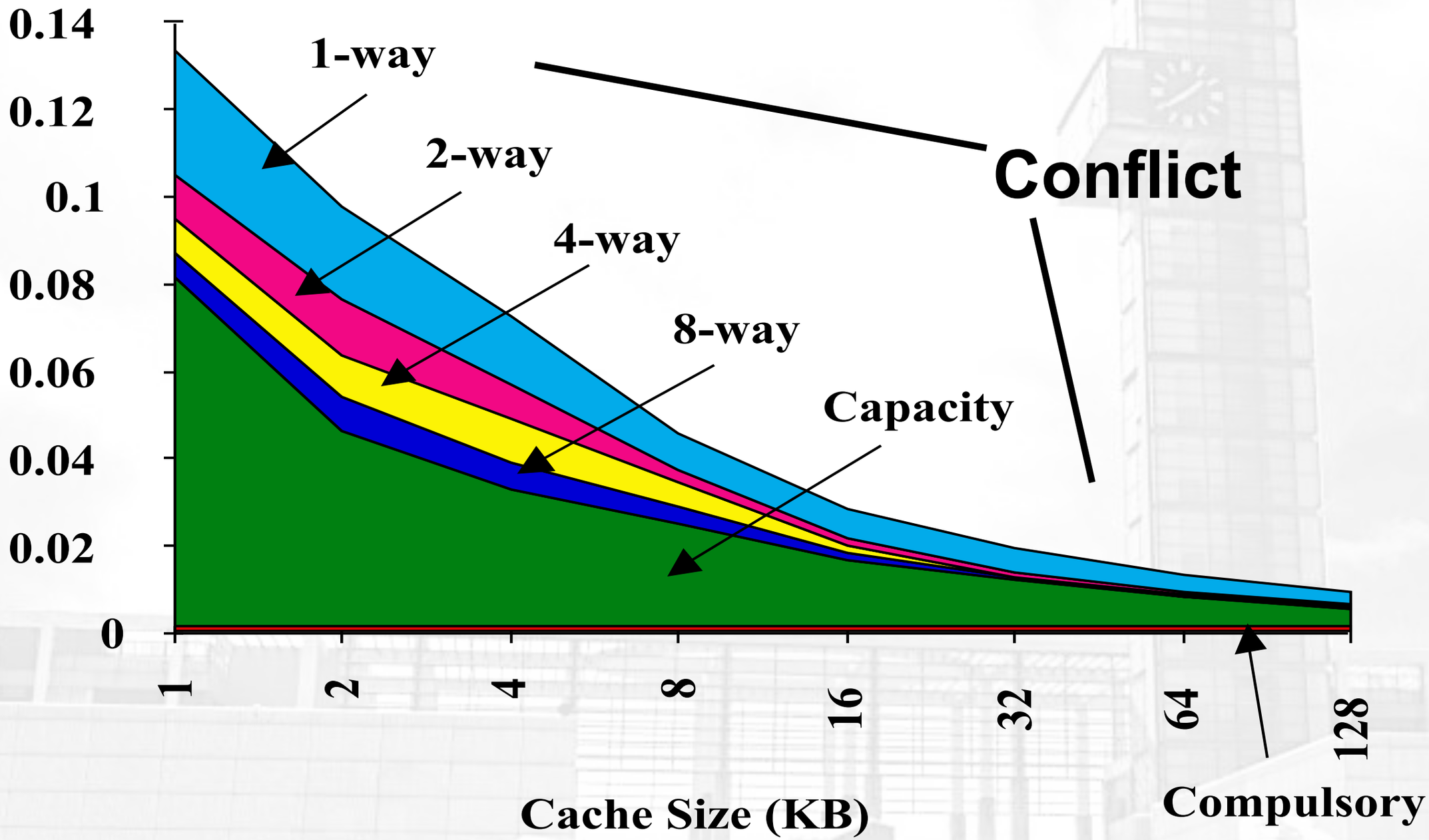
- **Cache命中(Hit)**
  - **CPU欲访问的数据已在缓存中，即可直接访问 Cache**
- **Cache非命中(Miss)**
  - **数据不在Cache，需要将主存数据调入Cache**
  - **CPU被阻塞(Blocking)，等！**
- **命中率(Hit Rate)**
- **非命中率(Miss Rate)**

Cache性能分析:Miss的分类4C

- Miss的4种类型(4C):
  - Compulsory（必然)
    - 冷启动、过程转移、首次引用等不可避免的miss...
  - Capacity（容量）
    - 容量不够造成的
  - Conflict（冲突）
    - Cache块冲突
      - 方案1：增加容量
      - 方案2：提升相联度
  - Coherence（相关）
    - 其它操作带来的结构相关（I/O操作等）

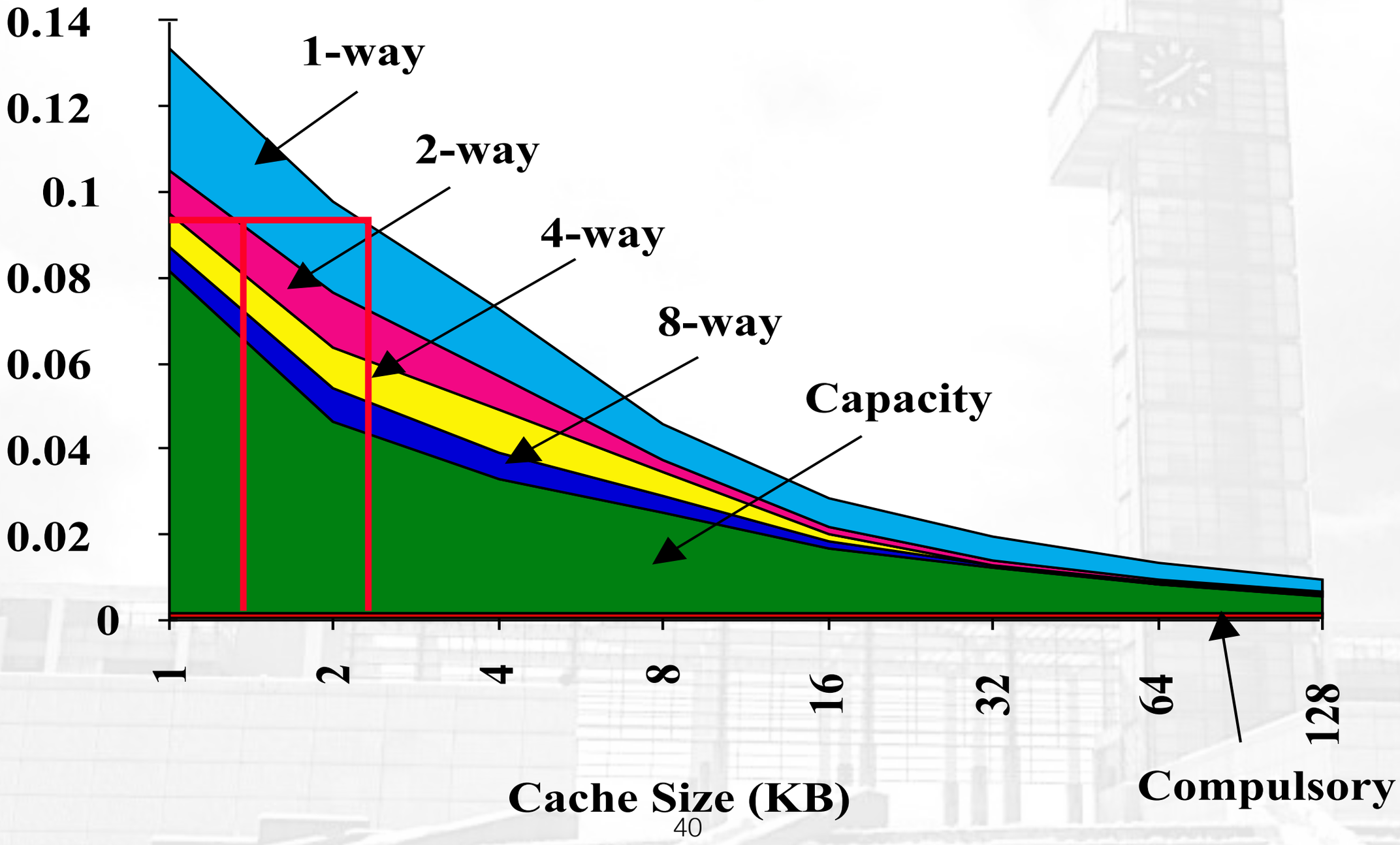
Cache性能分析:Miss的分类4C

- Conflict Miss随容量提升而降低



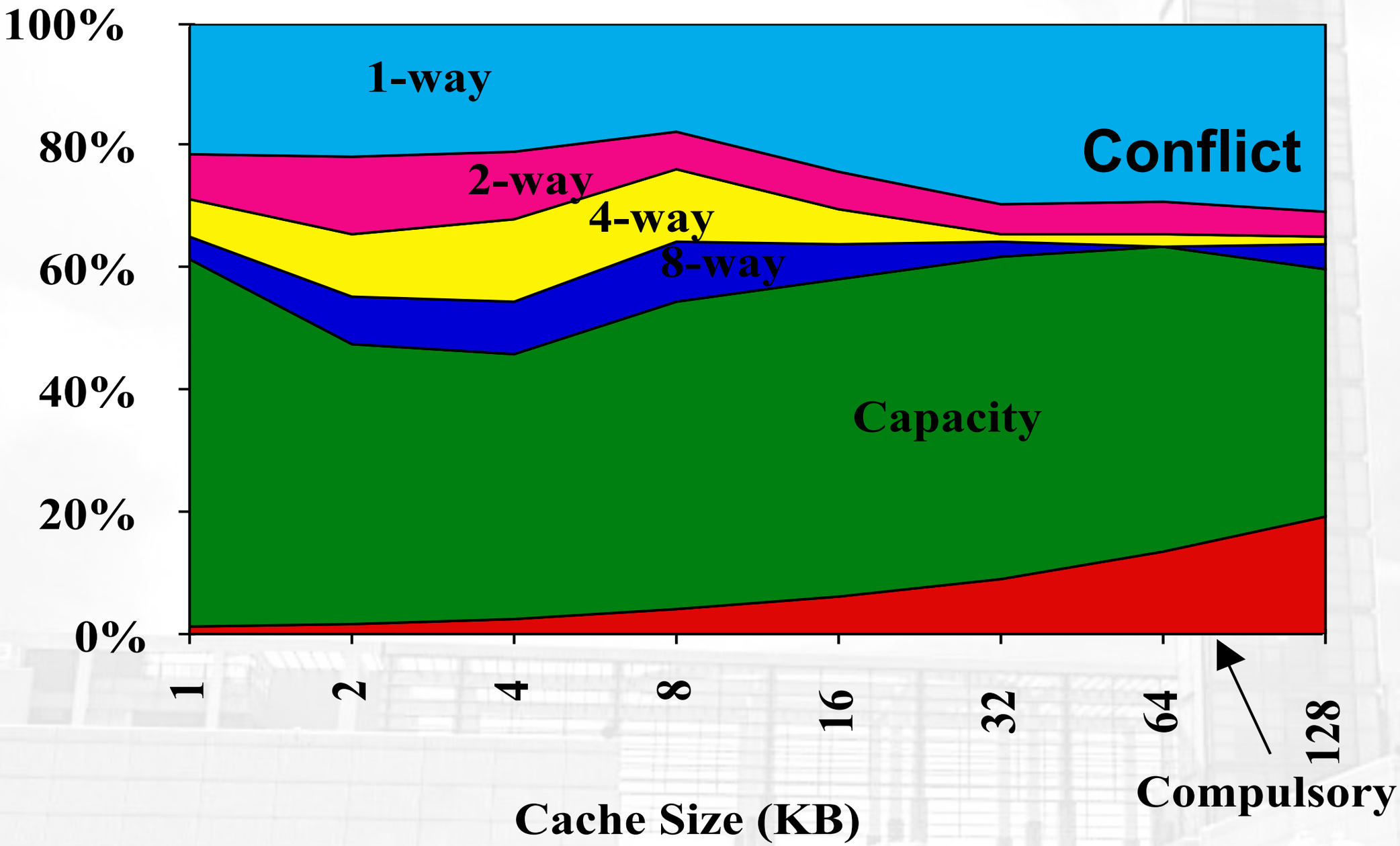
Cache性能分析:Miss的分类4C

- 多路组相联明显降低miss rate
  - 相同的miss rate，1-way组相联(直接映射)需要2倍的Cache size才能达到2-way组相联水平



Cache性能分析:Miss的分类4C

- 3种miss的比例



Cache性能分析与优化

1. 加速比

■ Cache-主存系统的平均访问周期T：

$$T = H \times TC + (1 - H) \times (TB + TC)$$

$$T = TC + (1-H) * TB$$

- Cache的访问周期为TC，主存的访问周期为TM，数据块调入Cache的块传输时间为TB，Cache的命中率为H。

■ Cache-主存系统的加速比SP：SP＝TM/T　<—(Old duration / new)

Cache性能分析与优化

例如，设Cache的速度是主存的5倍，命中率为 95%，则采用Cache后性能提升多少?

系统平均访问时间=*t+0.05*5t=1.25t*

性能提升=*5t/1.25t=4*倍

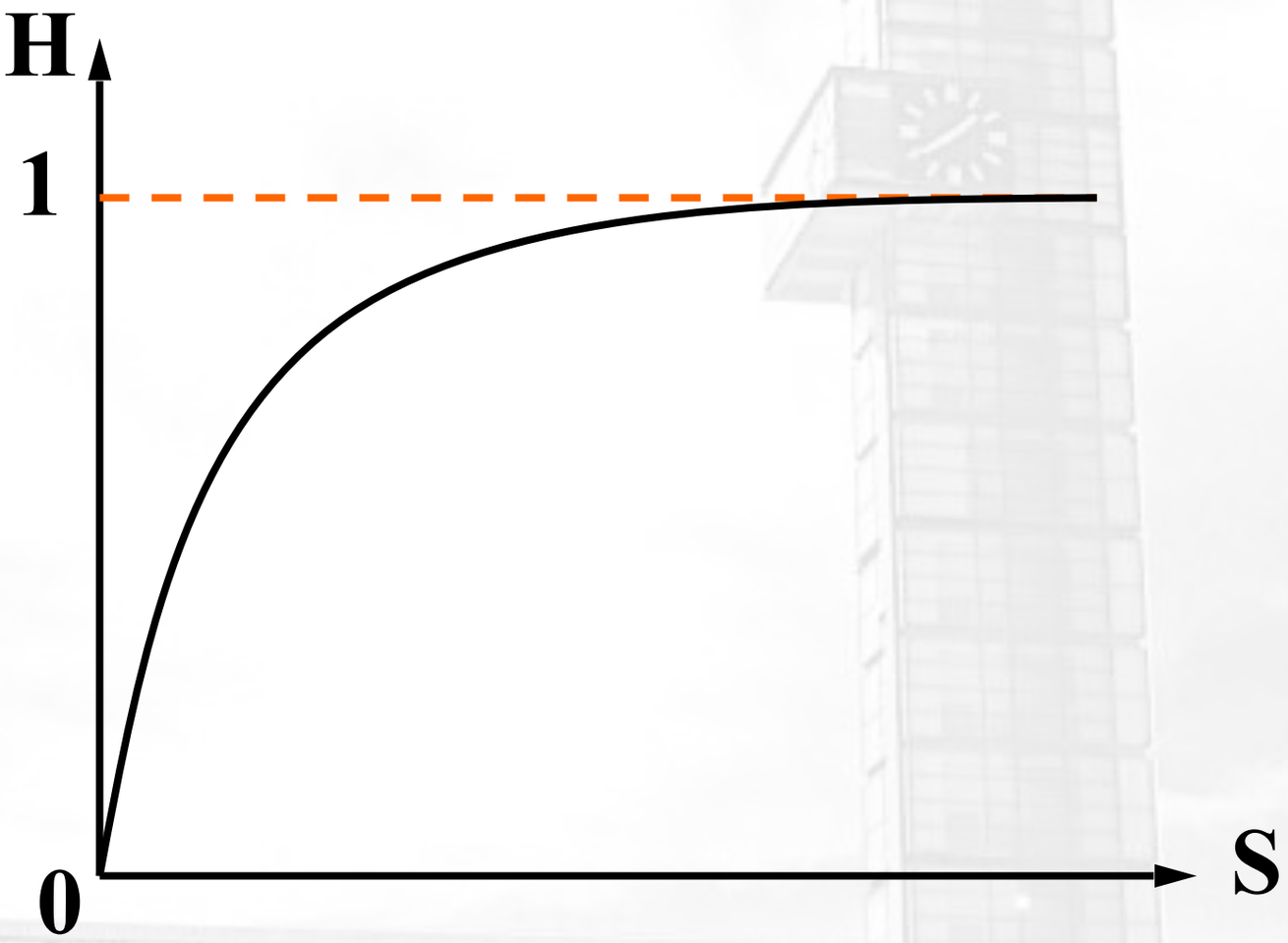
Cache性能分析与优化

- 如何提升Cache性能?
  - 容量
  - 块数/块大小
  - 多路组相联
  - 多级Cache
  - 指令/数据独立Cache
  - Victim Cache
  - 代码层面
  - …

Cache性能分析与优化

优化方案1：提升Cache容量

- 边际效率降低
- 成本问题…
- 制成/热设计…



Cache容量S与命中率H的关系

Cache性能分析与优化

存储系统成本

- **C = (C1×S1 + C2×S2)/(S1 + S2)**

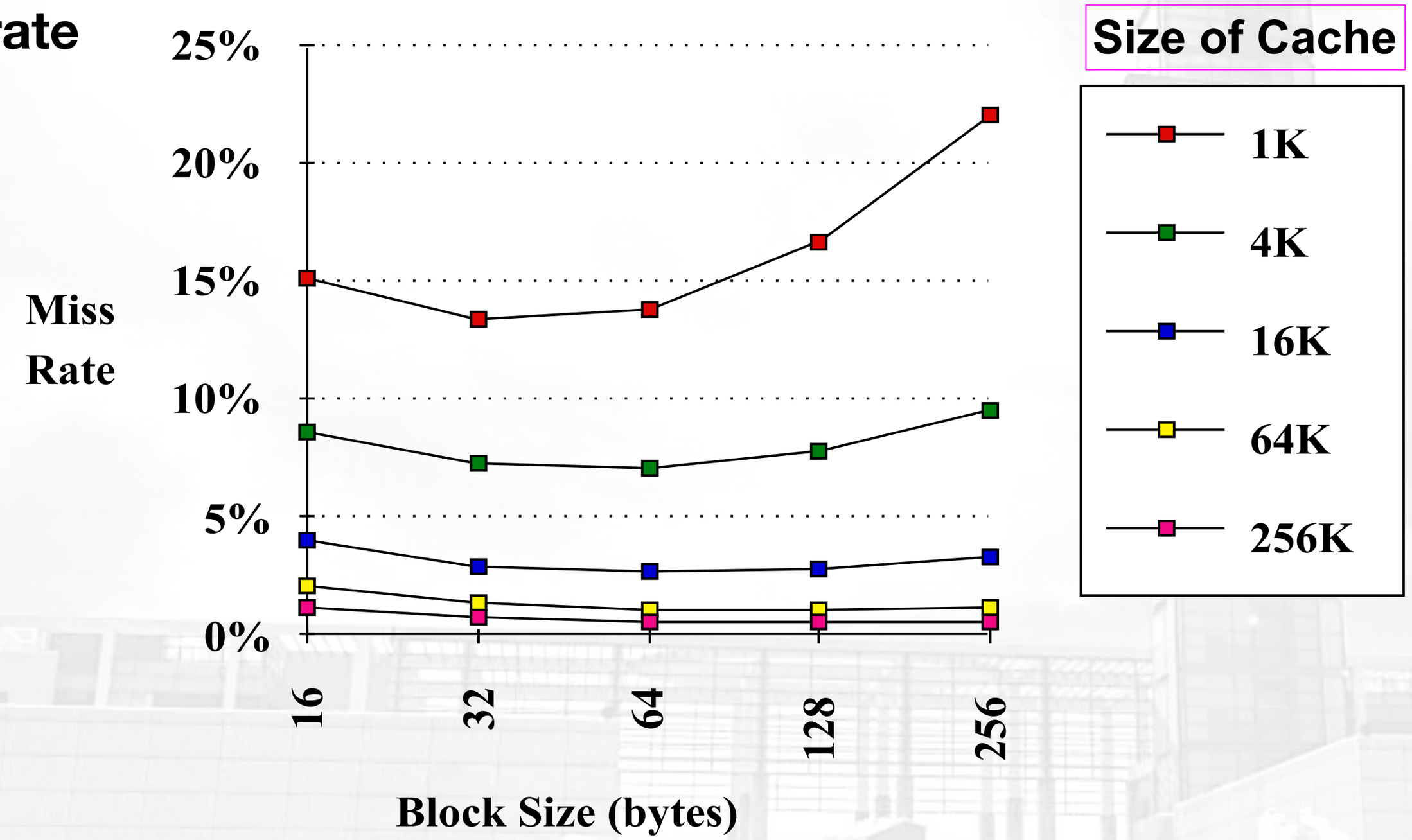
主存价格　主存容量　Cache容量　Cache价格

S1>>S2

- 尽管**Cache**的价格比主存高，但是当其容量很小时，存储器的平均价格**C**接近于主存的价格。
- 由于设置了**Cache**使**CPU**的访存速度接近**Cache**的速度，而使存储器的成本接近于主存的价格。

Cache性能分析与优化

优化方案2：利用时空局部性原理，通过提升块大小，降低miss rate



Cache性能分析与优化

优化方案2：利用时空局部性原理，通过提升块大小，降低miss rate

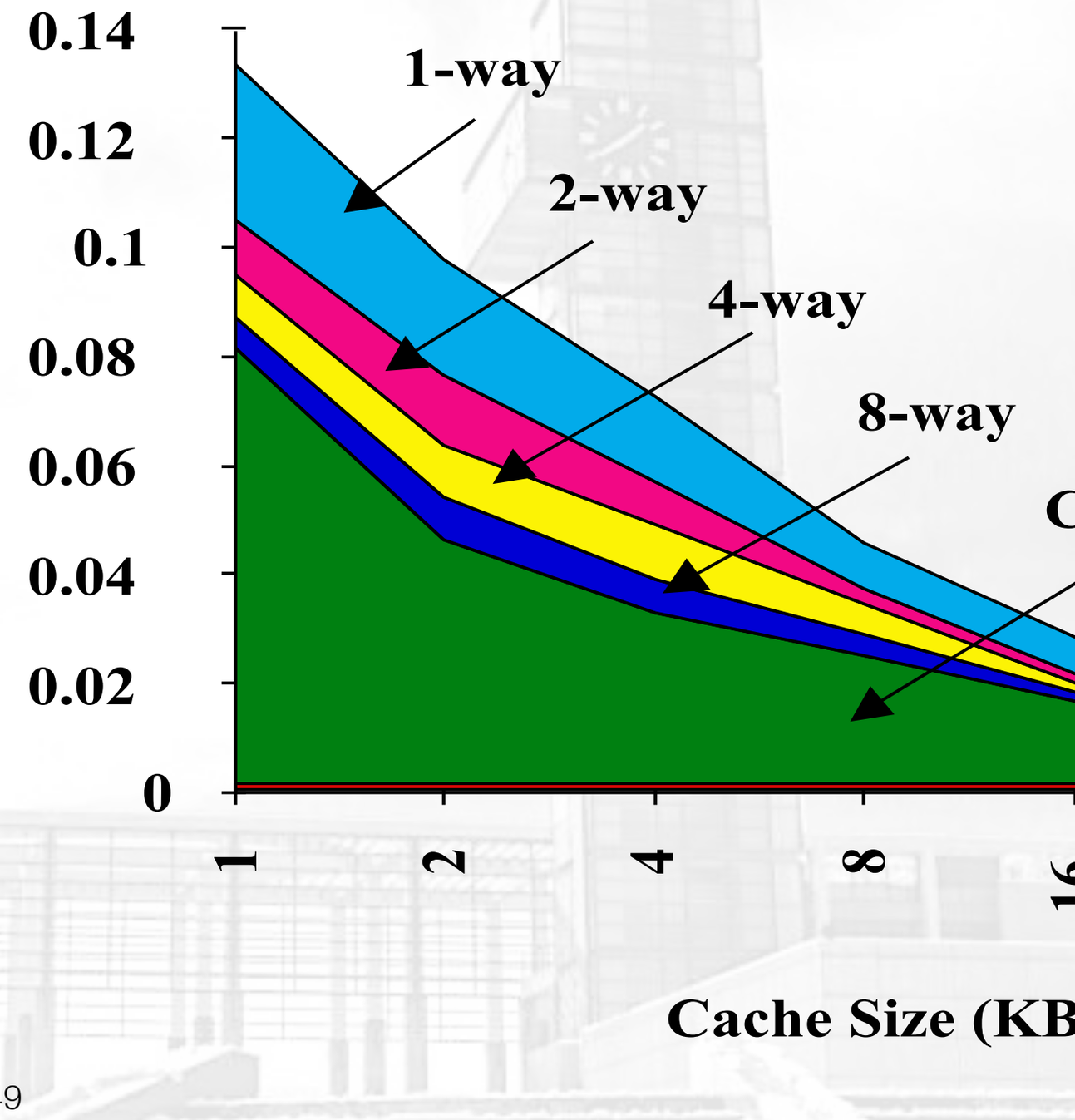
- 降低Compulsory必然miss rate
- 但太大的块有缺点
  - miss的代价大；
  - Conflict冲突导致的miss高

- 总结，差不多大就得了。

Cache性能分析与优化

优化方案3：提升组相联级别

- 级别高一些，还是好的...
  - 2:1规则，2路组相联只需要一半的Cache容量就可以达到1路组相联同等的miss rate/hit rate
  - 但级别提升的边际效率在下降
- 级别也不要太高
  - 相联比较复杂
  - 相联存取复杂—>提高TC时间



Cache性能分析与优化

- 例如—组典型数据…

Cache Size (KB)	Associativity 1-way	2-way	4-way	8-way
1	7.65	6.60	6.22	5.44
2	5.90	4.90	4.62	4.09
4	4.60	3.95	3.57	3.19
8	3.30	3.00	2.87	2.59
16	2.45	2.20	2.12	2.04
32	2.00	1.80	1.77	1.79
64	1.70	1.60	1.57	1.59
128	1.50	1.45	1.42	1.44

红色说明8路组相联已经没有性能提升

Cache性能分析与优化

优化方案4：多级Cache

- 两级**Cache**的总未命中率(总失效率)：

  总失效率＝(失效率)$_{第一级}$×(失效率)$_{第二级}$.....×(失效率)$_{第N级}$

- 【例】**10000**次访存，第一级**Cache**失效**400**次，第二级**Cache**失效**4**次。
  - (失效率)$_{第一级}$＝**400/10000**＝**4**%
  - (失效率)$_{第二级}$＝**4/400**＝**1**%
  - 利用两级**Cache**后总的失效率＝**0.04**%。

Cache性能分析与优化

5. 多级Cache的命中率

【例】访问内存需50ns, L1 1ns 10%失效率，L2 5ns 1%失效率，L3 10ns 0.2%失效率，求L1， L1+L2，L1+L2+L3构架下的平均访问时间。

L1 ： $T = 1\ ns + (0.1 \times 50\ ns) = 6\ ns$

L1+2 ： $T = 1\ ns + (0.1 \times [5\ ns + (0.01 \times 50\ ns)]) = 1.55\ ns$

L1+2+3 : $T = 1\ ns + (0.1 \times [5\ ns + (0.01 \times [10\ ns + (0.002 \times 50\ ns)])]) = 1.5001\ ns$

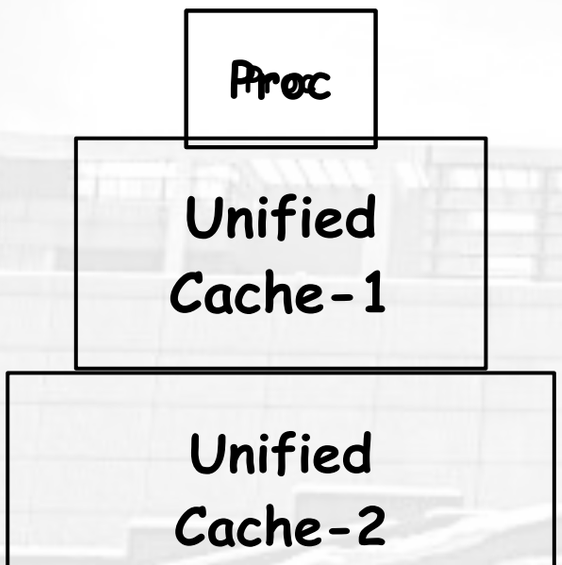
Cache性能分析与优化

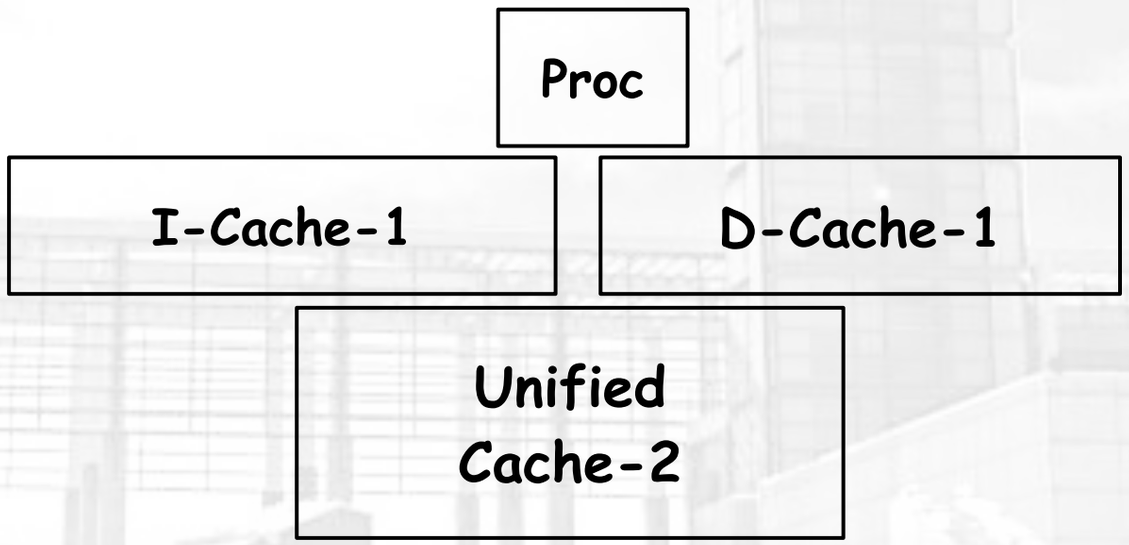
优化方案5：拆分出指令/数据独立Cache

- 例如
  - 16KB 指令/数据独立Cache
    - 指令 miss rate=0.64%, 数据 miss rate=6.47%
  - 32KB 统一Cache: miss rate=1.99%
  - 假设75%指令,25%数据， Hit time = 1, Miss time = 50

T独立=75%x(1+0.64%x50)+25%x(1+6.47%x50) = 2.05

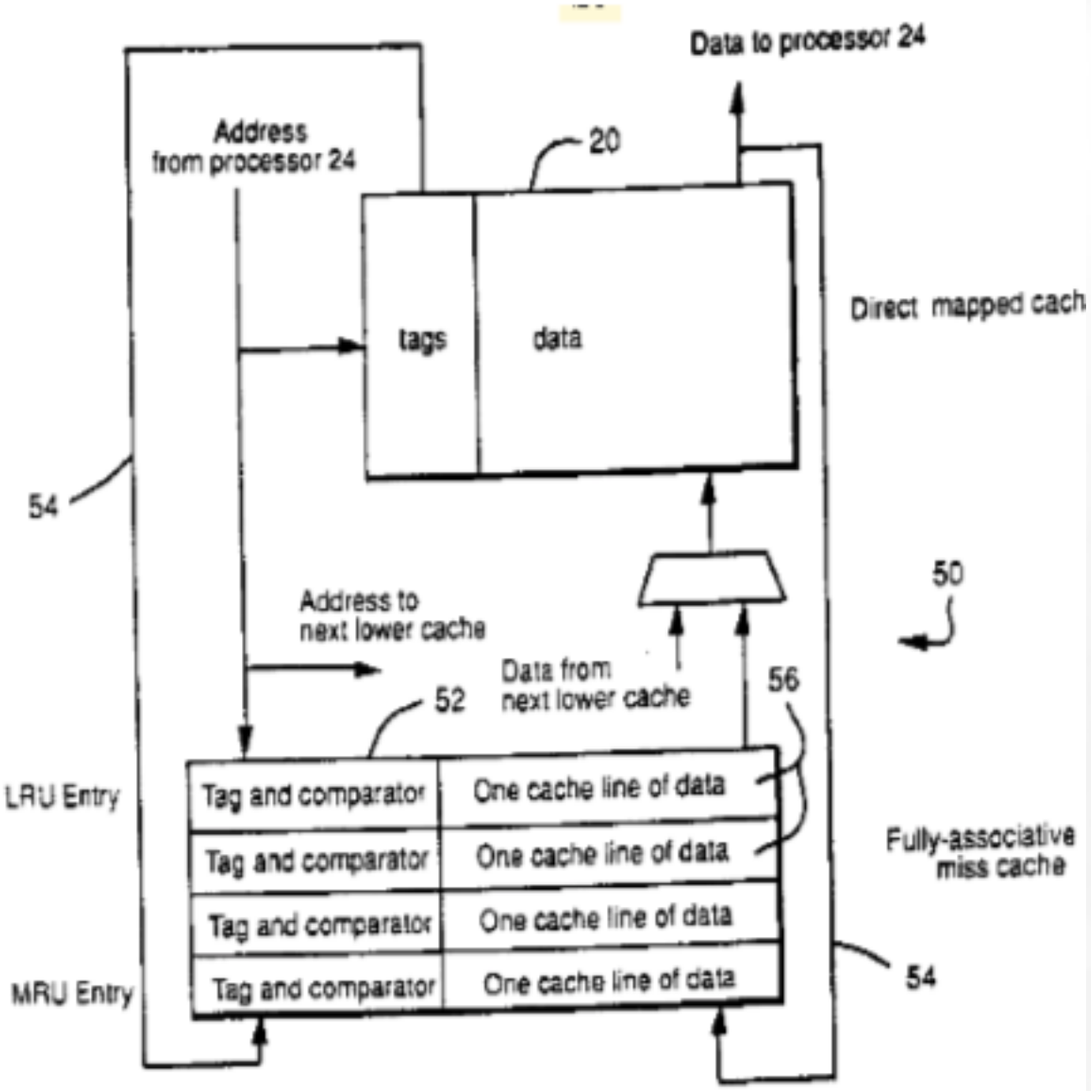
T统一=75%x(1+1.99%x50)+25%x(1+1+1.99%x50)= 2.24





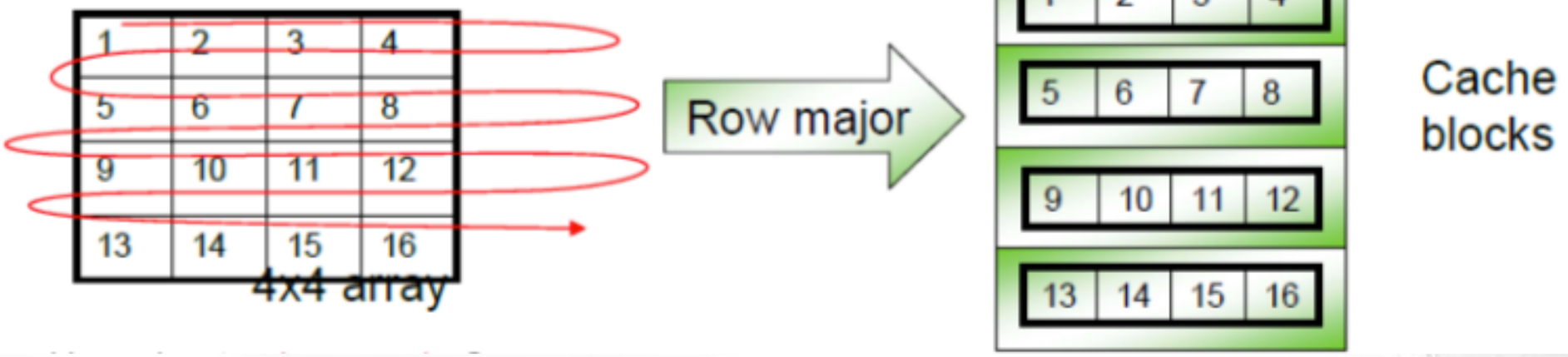
Cache性能分析与优化

- 优化方案6：Victim Cache
- Miss的代价很大，如何优化?
  - Victim Cache，在L1和L2/主存之间
- Victim Cache容量小，采用全相联
- 例如，4个块的Victim Cache可以降低4KB直接映射miss rate 20-90%。



Cache性能分析与优化

- 优化方案7：代码层面: 1) Row Major
- 注意循环：大部分计算和访存都在循环中发生
  - 按照数据对象在存储器中的存储顺序读取，最优
  - 否则，连续miss



- 原程序(Column Major)

```
a[100][5000]=...//初始化
for(j=0; j<5000; j=j+1) {
  for(i=0; i<100; i=i+1) {
    a[i][j] = 2 * a[i][j]; 每次都不命中
  }
}
```

- 优化后(Row Major)

```
a[100][5000]=...//初始化
for(i=0; i<100; i=i+1) {
  for(j=0; j<5000; j=j+1) {
    a[i][j] = 2 * a[i][j]; 可连续命中若干次[cache行大小]
  }
}
```

Cache性能分析与优化

- 优化方案7：代码层面: 2) 循环合并

```
for(i := 0; i < N; i := i+1)
    for(j := 0; j < N; j := j+1)
        a[i][j] := 1 / b[i][j] * c[i][j];
for(i := 0; i < N; i := i+1)
    for(j := 0; j < N; j := j+1)
    d[i][j] := a[i][j] + c[i][j];
```

- 原程序中a和c访问两次
- 有可能不在cache中

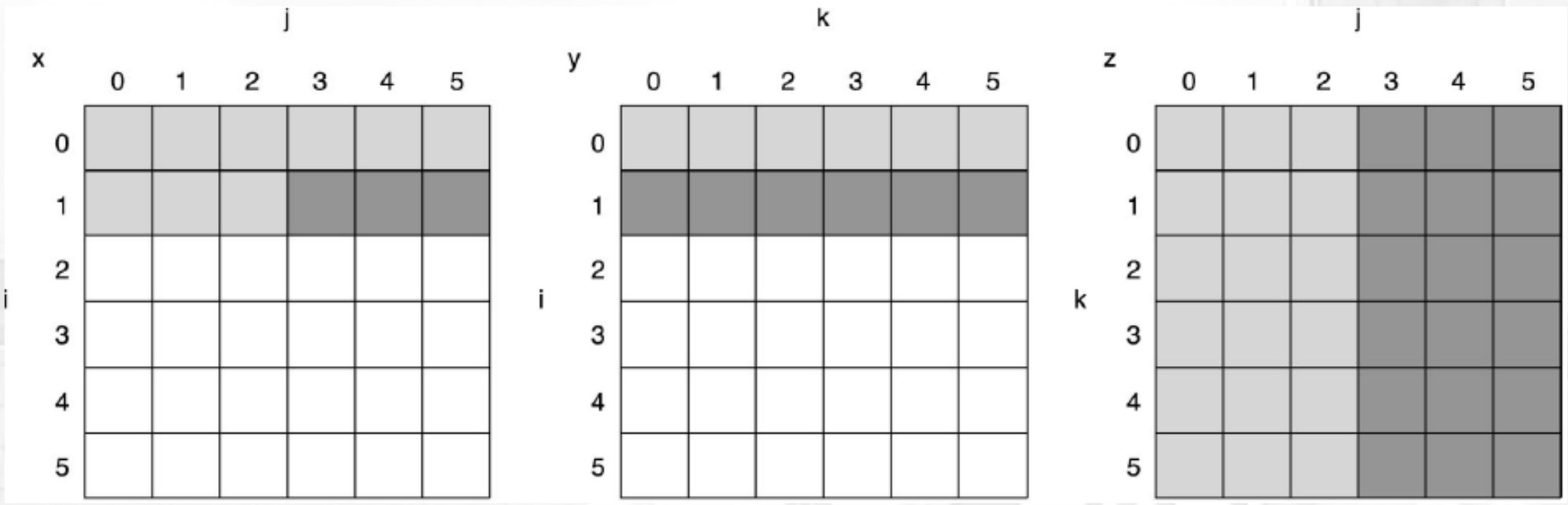
```
for(i := 0; i < N; i := i+1)
    for(j := 0; j < N; j := j+1){
        a[i][j] := 1 / b[i][j] * c[i][j];
        d[i][j] := a[i][j] + c[i][j];}
```

优化后a和c第2次访问都在cache里

Cache性能分析与优化

- 优化方案7：代码层面: 3) 分块运算
- 例如，矩阵.*运算
  - 当3个矩阵无法全放在cache时，严重的Capacity Miss

```
for(i := 0; i < N; i := i+1)
    for(j := 0; j < N; j := j+1){
        r := 0;
        for(k := 0; k < N; k := k+1)
            r := r + y[i][k] * z[k][j];
x[i][j] := r;}
```

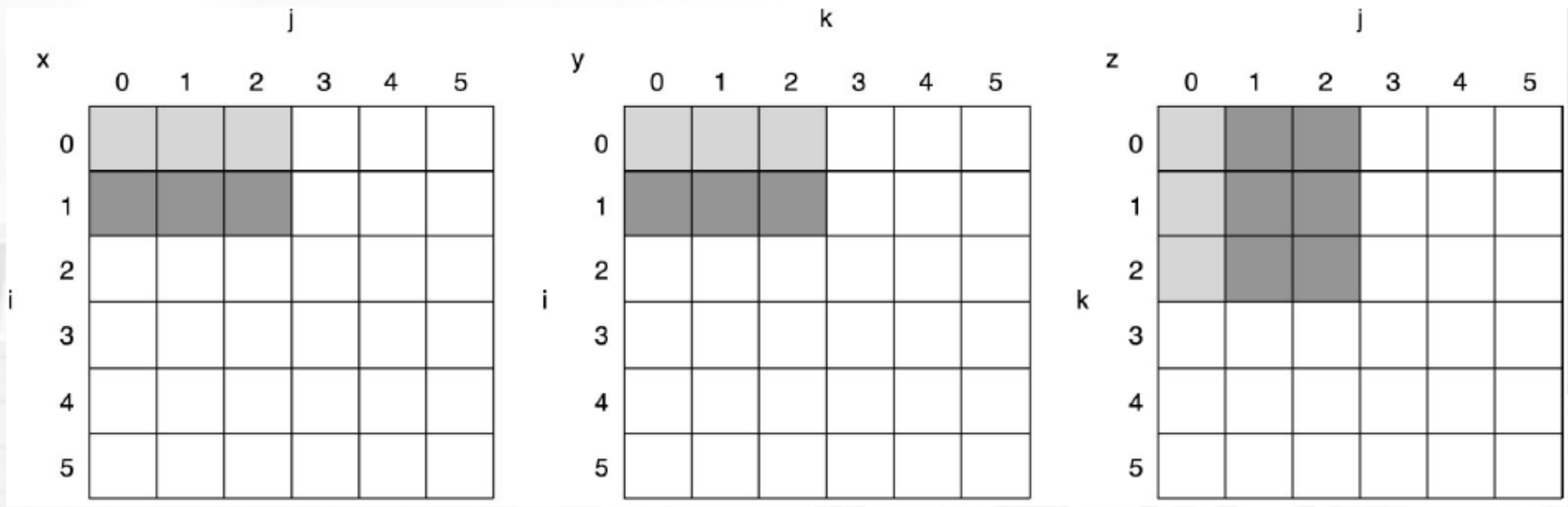


Cache性能分析与优化

- 优化方案7：代码层面: 3) 分块运算
- 例如，矩阵.*运算
  - 当3个矩阵无法全放在cache时，严重的Capacity Miss
  - 把矩阵分成小块执行，充分避免Capacity Miss

After:

```
for(jj := 0; jj < N; jj := jj+B)
for(kk := 0; kk < N; kk := kk+B)
for(i := 0; i < N; i := i+1)
for(j := jj; j < min(jj+B-1,N); j := j+1){
    r := 0;
    for(k := kk; k < min(kk+B-1,N); k := k+1)
        r := r + y[i][k] * z[k][j];
    x[i][j] := x[i][j] + r;}
```

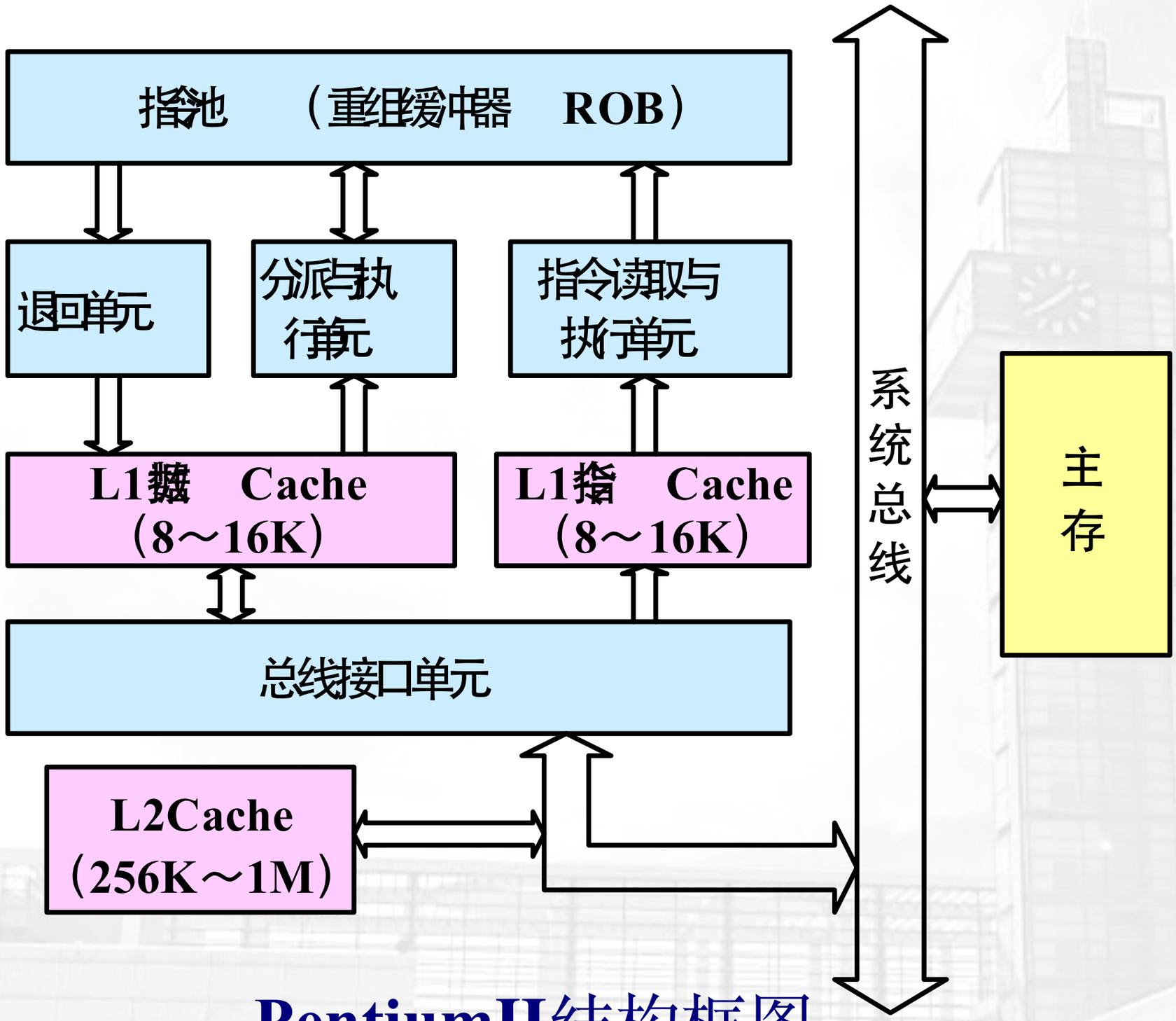


7.3.5 Pentium的Cache

Pentium II 处理器

- 两级**Cache**
  - **L1**：**8～16KB**，**CPU**内部集成
    - 数据 **Cache**：可随机读/写，双端口存储器
    - 指令 **Cache**：只读
  - **L2**：容量大，在**CPU**外部
- 地址映射：组相联。
  - 对于**8K**的**L1 Cache**：
    - **128**组，每组**2**块(行)，每块(行)**32B**。
- 替换算法：**LRU**

7.3.5 Pentium的Cache



PentiumII结构框图

- **Pentium 4 Cache**
  - L1 caches: 64 byte lines，4 way set associative
  - L2 cache: 128 byte lines，8 way set associative
  - L3 cache : on chip

